Arquitecturas de software y patrones de diseño

Arquitecturas de software

1. Arquitectura en capas

- Divide la aplicación en capas (presentación, lógica de negocio, datos) para que cada parte tenga una responsabilidad.
- Ejemplo: una app web con frontend, backend y base de datos.



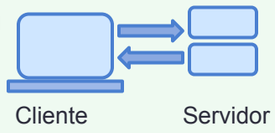
2. Arquitectura monolítica

- Todo el sistema está en un solo proyecto y se despliega junto.
- Es más simple, pero menos flexible para escalar.
- Ejemplo: una app pequeña como un blog.



3. Arquitectura cliente-servidor

- El cliente (usuario) solicita información y el servidor la procesa y responde.
- Permite centralizar los datos y recursos.
- Ejemplo: un navegador (cliente) que se conecta a un servidor web.



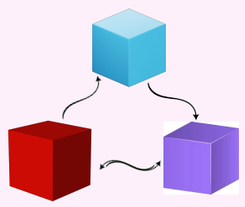
4. Arquitectura MVC

- Separa la lógica en 3 partes: modelo, vista y controlador.
- Facilita el mantenimiento y la organización del código.
- Ejemplo: una página web con MVC en frameworks como Spring o Laravel.



5. Arquitectura de microservicios

- Divide la aplicación en servicios pequeños e independientes.
- Cada servicio se comunica por APIs.
- Permite escalar y actualizar partes sin afectar todo el sistema.
- Ejemplo: una app de e-commerce con servicios de usuarios, pedidos y pagos.



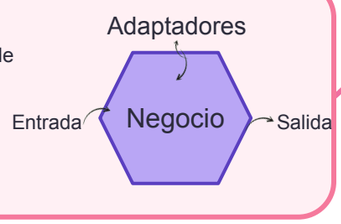
6. Arquitectura orientada a servicios (SOA)

- Organiza el sistema en servicios reutilizables que se comunican entre sí.
- Se enfoca en la integración entre aplicaciones.
- Ejemplo: empresas que comparten servicios como facturación o inventario.



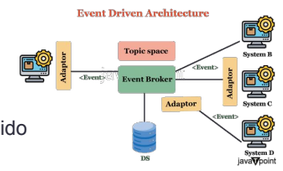
7. Arquitectura hexagonal

- Aísla la lógica de negocio de los detalles externos (base de datos, APIs, interfaz).
- Facilita las pruebas y el cambio de tecnologías.
- Ejemplo: una API que se conecta a la base de datos a través de un puerto (adapter).



8. Arquitectura dirigida por eventos

- Los componentes se comunican produciendo y reaccionando a eventos.
- Permite sistemas desacoplados y asíncronos.
- Ejemplo: una tienda en línea que genera el evento “pedido creado” para avisar a inventario y pagos.



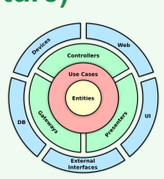
9. Arquitectura serverless

- El proveedor en la nube administra los servidores; solo se escribe el código de las funciones.
- Se paga por uso y escala automáticamente.
- Ejemplo: una función que redimensiona imágenes al subirlas.



10. Arquitectura limpia (Clean Architecture)

- Organiza el código en círculos concéntricos; las dependencias apuntan hacia las reglas de negocio.
- Es independiente de frameworks, base de datos e interfaz.
- Ejemplo: casos de uso que funcionan igual con una app web o móvil.



Patrones de diseño

1. Singleton y Factory Method

Singleton
- Garantiza una sola instancia de una clase en todo el sistema.
- Ejemplo: conexión a la base de datos.

Factory Method
- Crea objetos sin especificar la clase exacta.
- Ejemplo: una fábrica de formas de pago.



2. Abstract Factory, Builder y Prototype

Abstract Factory
- Crea familias de objetos relacionados.
- Ejemplo: interfaz gráfica con tema claro/oscuro.

Builder
- Construye objetos paso a paso.
- Ejemplo: crear un auto con varias partes.

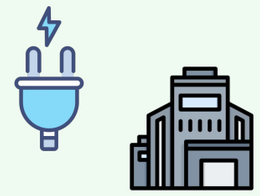
Prototype
- Clona objetos existentes.
- Ejemplo: duplicar un documento.



3. Adapter y Facade

Adapter
- Hace que una interfaz incompatible funcione con otra.
- Ejemplo: un cargador de celular para otro tipo de enchufe.

Facade
- Simplifica la interacción con un sistema complejo.
- Ejemplo: una clase que agrupa varias APIs.



4. Decorator, Proxy y Bridge

Decorator
- Agrega funcionalidades sin modificar la clase original.
- Ejemplo: añadir “extra” a un café.

Proxy
- Controla el acceso a un objeto.
- Ejemplo: verificar permisos antes de usar un servicio.

Bridge
- Separa la abstracción de su implementación.
- Ejemplo: un control remoto que funciona con distintos dispositivos.



5. Composite y Flyweight

Composite
- Trata objetos individuales y grupos de forma igual.
- Ejemplo: carpetas y archivos en un sistema.

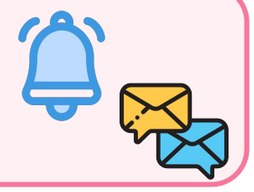
Flyweight
- Comparte datos para ahorrar memoria.
- Ejemplo: muchos árboles con el mismo tipo de hoja.



6. Observer y Mediator

Observer
- Notifica a varios objetos cuando cambia un estado.
- Ejemplo: un botón que actualiza la pantalla.

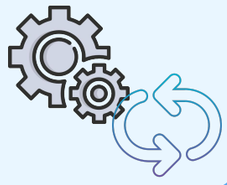
Mediator
- Centraliza la comunicación entre objetos.
- Ejemplo: un chat que gestiona los mensajes.



7. Strategy y State

Strategy
- Permite cambiar el comportamiento según la necesidad.
- Ejemplo: diferentes formas de pago.

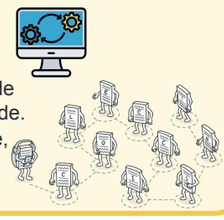
State
- Cambia el comportamiento según el estado del objeto.
- Ejemplo: un pedido (pendiente, enviado, entregado).



8. Command y Chain of Responsibility

Command
- Convierte una acción en un objeto que se puede ejecutar, guardar o deshacer.
- Ejemplo: el botón “deshacer” de un editor.

Chain of Responsibility
- Pasa una solicitud por una cadena de manejadores hasta que uno la atiende.
- Ejemplo: aprobar un gasto entre jefe, gerente y director.



9. Iterator, Memento e Interpreter

Iterator
- Recorre los elementos de una colección sin exponer su estructura.
- Ejemplo: recorrer una lista de productos.

Memento
- Guarda y restaura el estado anterior de un objeto.
- Ejemplo: deshacer cambios en un documento.

Interpreter
- Define una gramática y evalúa expresiones.
- Ejemplo: calcular una fórmula escrita como texto.



10. Template Method y Visitor

Template Method
- Define el esqueleto de un algoritmo y deja pasos a las subclases.
- Ejemplo: preparar bebidas (hervir agua, preparar, servir).

Visitor
- Agrega operaciones a objetos sin modificarlos.
- Ejemplo: exportar distintas figuras a PDF o XML.

